

Write a menu driven program to implement following operations on the singly linked list. (a) Insert a node at the front of the linked list. (b) Insert a node at the end of the linked list. (c) Insert a node such that linked list is in ascending order.(according to info. Field) (d) Delete a first node of the linked list. (e) Delete a node before specified position. (f) Delete a node after specified position.

```
#include<stdio.h>
#include<stdlib.h>

struct node
{
    int info;
    struct node *next;
};

struct node *head = NULL;

// Insert at Front
void insert_front()
{
    struct node *newnode;
    newnode = (struct node*)malloc(sizeof(struct node));

    printf("Enter value: ");
    scanf("%d",&newnode->info);

    newnode->next = head;
    head = newnode;

    printf("Inserted at front.\n");
}

// Insert at End
void insert_end()
{
    struct node *newnode, *temp;
    newnode = (struct node*)malloc(sizeof(struct node));

    printf("Enter value: ");
    scanf("%d",&newnode->info);

    newnode->next = NULL;

    if(head == NULL)
        head = newnode;
    else
    {
        temp = head;
        while(temp->next != NULL)
        {
```

```

        temp = temp->next;
    }
    temp->next = newnode;
}

printf("Inserted at end.\n");
}

// Insert in Ascending Order
void insert_sorted()
{
    struct node *newnode, *temp;
    newnode = (struct node*)malloc(sizeof(struct node));

    printf("Enter value: ");
    scanf("%d",&newnode->info);

    if(head == NULL || newnode->info < head->info)
    {
        newnode->next = head;
        head = newnode;
    }
    else
    {
        temp = head;
        while(temp->next != NULL && temp->next->info < newnode->info)
        {
            temp = temp->next;
        }
        newnode->next = temp->next;
        temp->next = newnode;
    }

    printf("Inserted in ascending order.\n");
}

// Delete First Node
void delete_first()
{
    struct node *temp;

    if(head == NULL)
    {
        printf("List is empty.\n");
        return;
    }

    temp = head;
    head = head->next;

```

```

    free(temp);

    printf("First node deleted.\n");
}

// Delete Before Position
void delete_before_pos()
{
    int pos,i;
    struct node *temp, *prev;

    printf("Enter position: ");
    scanf("%d",&pos);

    if(pos <= 2 || head == NULL || head->next == NULL)
    {
        printf("Invalid position.\n");
        return;
    }

    temp = head;
    for(i=1;i<pos-2;i++)
    {
        if(temp->next == NULL)
            return;
        temp = temp->next;
    }

    prev = temp->next;
    temp->next = prev->next;
    free(prev);

    printf("Node before position deleted.\n");
}

// Delete After Position
void delete_after_pos()
{
    int pos,i;
    struct node *temp, *del;

    printf("Enter position: ");
    scanf("%d",&pos);

    temp = head;
    for(i=1;i<pos;i++)
    {
        if(temp == NULL)
            return;

```

```

    temp = temp->next;
}

if(temp == NULL || temp->next == NULL)
{
    printf("Invalid position.\n");
    return;
}

del = temp->next;
temp->next = del->next;
free(del);

printf("Node after position deleted.\n");
}

// Display
void display()
{
    struct node *temp = head;

    if(temp == NULL)
    {
        printf("List empty.\n");
        return;
    }

    while(temp != NULL)
    {
        printf("%d -> ", temp->info);
        temp = temp->next;
    }
    printf("NULL\n");
}

// Main
int main()
{
    int choice;

    while(1)
    {
        printf("\n--- MENU ---\n");
        printf("1. Insert at Front\n");
        printf("2. Insert at End\n");
        printf("3. Insert in Ascending Order\n");
        printf("4. Delete First Node\n");
        printf("5. Delete Before Position\n");
        printf("6. Delete After Position\n");
    }
}

```

```
printf("7. Display\n");  
printf("8. Exit\n");
```

```
printf("Enter choice: ");  
scanf("%d",&choice);
```

```
switch(choice)  
{  
    case 1: insert_front(); break;  
    case 2: insert_end(); break;  
    case 3: insert_sorted(); break;  
    case 4: delete_first(); break;  
    case 5: delete_before_pos(); break;  
    case 6: delete_after_pos(); break;  
    case 7: display(); break;  
    case 8: exit(0);  
    default: printf("Invalid choice!\n");  
}
```

```
}
```

```
}
```